MATH 4FM3 — Final Project Roadmap

Deep Q-Learning for Nash Equilibria: Nash-DQN

Based on arXiv:1904.10554v2 (Casgrain, Ning & Jaimungal, 2022)

McMaster University · Department of Mathematics & Statistics

4-Week Research-Grade Project Sprint · MATH 4FM3

Student	Role	Specialisation	Primary Ownership
A	Theory Lead 1	Math finance, SDEs, stochastic calculus	Price/impact dynamics, Nash derivation
B	Theory Lead 2	Stochastic control, Bellman, optimisation	Q-function structure, reward, benchmarks
C	Practice Lead 1	RL algorithms, PyTorch, gym environments	Simulator, replay buffer, training loop
D	Practice Lead 2	ML engineering, neural networks	Network arch., Nash-DQN solver, plots

Contents

1 Mathematical Model of the Paper (arXiv:1904.10554v2) 2
1.1 Stochastic Game Setup (Section 2 of Paper) . . . 2
1.2 Optimality Conditions and Nash-Bellman Equation (Section 3) . . . 2
1.3 Locally Linear-Quadratic Q-Function (Section 4) . . . 2
1.4 Electronic Trading Application (Section 6) . . . 3
1.5 Nash-DQN Actor-Critic Algorithm (Algorithm 5.1) . . . 4

2 Theory-to-Practice Mapping 5

3 Technical Specifications for 2-Agent Simplified Implementation 6
3.1 State Vector . . . 6
3.2 Action Space . . . 6
3.3 Network Architecture . . . 6
3.4 Q-Value Computation . . . 6
3.5 Training Loss . . . 6
3.6 Stability Specifications . . . 6

4 Four-Week Detailed Roadmap 7

5 Theory vs Practice Group Responsibilities 11

6 Recommended Resources 11

7 Grading Checklist & Risk Mitigation 12

Mathematical Model of the Paper (arXiv:1904.10554v2)

All notation follows the paper directly. Students A and B must master this section before Week 2 begins and are responsible for verifying that every formula below is faithfully implemented in code.

Stochastic Game Setup (Section 2 of Paper)

Consider a stochastic game with N agents, $i \in \mathcal{N} := \{1, 2, \ldots, N\}$. The game state at time t is $x_t \in \mathcal{X}$, where $\mathcal{X}$ is a separable Banach space. Each agent i selects action $u_{i,t} \in \mathcal{U}_i$ (also a Banach space). The MDP state-transition kernel is $p(x_{t+1} \mid x_t, u_t)$.

Agent i's objective (equation (2.1) in the paper):

$$R_i\big(x; \pi_i, \pi_{-i}\big) \;=\; \mathbb{E}\left[\sum_{t=0}^{\infty} \gamma_i^t \, r_i(x_t, \pi_i(x_t), \pi_{-i}(x_t))\right], \qquad \gamma_i \in (0,1). \tag{1}$$

Definition 1 (Nash Equilibrium, eq. (2.2)). *A collection of policies $\pi^* = \{\pi_i^*\}_{i\in\mathcal{N}}$ is a **Nash equilibrium** if*

$$R_i\big(x; \pi_i, \pi_{-i}^*\big) \;\le\; R_i\big(x; \pi_i^*, \pi_{-i}^*\big) \qquad \forall \text{ admissible } \pi_i, \; \forall\, i \in \mathcal{N}. \tag{2}$$

Optimality Conditions and Nash-Bellman Equation (Section 3)

Q-Function (eq. (3.2))

Define the Nash state-action value (Q-)function $Q(x;u) = (Q_i(x; u_i, u_{-i}))_{i\in\mathcal{N}}$ by

$$Q(x;u) \;=\; r(x;u) \;+\; \gamma_i \, \mathbb{E}_{x'\sim p(\cdot|x,u)}\big[V(x')\big], \tag{3}$$

where $r(x,u) := (r_i(x, u_i, u_{-i}))_{i\in\mathcal{N}}$ and $V(x) = (V_i(x))_{i\in\mathcal{N}}$ is the stacked value function.

Definition 2 (Nash Operator, Def. 3.1). *For a collection of concave functions $f(u) = (f_i(u_i, u_{-i}))_{i\in\mathcal{N}}$, the **Nash operator** $\mathrm{Nash}_{u\in\mathcal{U}}\ f(u) \mapsto f(u^*)$ maps to the Nash equilibrium value $u^* \in \mathbb{R}^N$, where u^* satisfies*

$$f_i(\omega_i, u_{-i}^*) \;\le\; f_i(u_i^*, u_{-i}^*) \qquad \forall\, \omega_i \in \mathcal{U}_i, \; \forall\, i \in \mathcal{N}. \tag{4}$$

Nash-Bellman Equation (eq. (3.4))

$$V(x) \;=\; \mathrm{Nash}_{u\in\mathcal{U}}\, Q(x;u) \;=\; \mathrm{Nash}_{u\in\mathcal{U}}\big\{r(x;u) + \gamma_i \, \mathbb{E}_{x'\sim p(\cdot|x,u)}\big[V(x')\big]\big\}. \tag{5}$$

Hence $\pi^* = \mathrm{Nash}_{u\in\mathcal{U}}\, Q(x;u)$. Identifying Q satisfying (5) is sufficient to recover the Nash equilibrium.

Locally Linear-Quadratic Q-Function (Section 4)

Decomposition (eq. (4.3))

$$\hat{Q}^\theta(x;u) \;=\; \hat{V}^\theta(x) \;+\; \mathcal{A}^\theta(x;u). \tag{6}$$

$\hat{V}^\theta$ is the value-function network; $\mathcal{A}^\theta$ is the advantage network with a locally linear-quadratic form.

Advantage Function (eq. (4.4))

For each agent i and state x,

$$\mathcal{A}_i^\theta(x;u) \;=\; -\begin{pmatrix} u_i - \mu_i^\theta(x) \\ u_{-i} - \mu_{-i}^\theta(x) \end{pmatrix}^\top P_i^\theta(x) \begin{pmatrix} u_i - \mu_i^\theta(x) \\ u_{-i} - \mu_{-i}^\theta(x) \end{pmatrix} + \big(u_{-i} - \mu_{-i}^\theta(x)\big)^\top \Psi_i^\theta(x), \tag{7}$$

where the block matrix is

$$P_i^\theta(x) \;:=\; \begin{pmatrix} P_{11,i}^\theta(x) & P_{12,i}^\theta(x) \\ P_{21,i}^\theta(x) & P_{22,i}^\theta(x) \end{pmatrix}, \tag{8}$$

with $P_{11,i}^\theta(x) \succ 0$ (positive-definite) for all x, ensuring concavity of $\hat{Q}_i$ in u_i. The positive-definiteness is enforced via Cholesky decomposition: $P_{11,i}^\theta(x) = L_{11,i}^\theta(x)\,\big(L_{11,i}^\theta(x)\big)^\top$.

Nash Equilibrium in Closed Form (eq. (4.6))

Under model (7), the advantage function is zero at $u^*(x) = \mu^\theta(x)$, giving directly:

$$\mu^\theta(x) \;=\; \arg\max_{u\in\mathcal{U}} \hat{Q}^\theta(x;u) \;=\; u^*(x). \tag{9}$$

The value function satisfies $\hat{V}^\theta(x) = \mathrm{Nash}_u\ \hat{Q}^\theta(x;u)$.

Loss Function (eqs. (4.7a)–(4.7b))

For each observed transition $y_m = (x_m, u_m, x'_m)$ the sample loss is

$$\mathcal{L}_m(\theta) \;=\; \Big\| \hat{V}^\theta(x_m) + \mathcal{A}^\theta(x_m;\,u_m) - r(x_m;\,u_m) - \gamma_i\,\hat{V}^\theta(x'_m) \Big\|^2, \tag{10}$$

and the total loss minimised over M mini-batch samples is

$$\mathcal{L}(\theta) \;=\; \frac{1}{M}\sum_{m=1}^{M} \mathcal{L}_m(\theta). \tag{11}$$

Label-Invariant Simplification (eq. (4.9))

When agents are label-invariant with $d_j = d$ for all j, the advantage simplifies to

$$\mathcal{A}_i^\theta(x;u) \;=\; -\Big\|u_i - \mu_i^\theta\Big\|^2_{P^\theta_{11,i}} - \sum_{j\neq i}\Big\langle u_i - \mu_i^\theta,\, u_j - \mu_j^\theta \Big\rangle_{P^\theta_{12,i}} - \sum_{j\neq i}\Big\|u_j - \mu_j^\theta\Big\|^2_{P^\theta_{22,i}} + \sum_{j\neq i}(u_j - \mu_j^\theta)^\top \psi^\theta(x), \tag{12}$$

reducing the parameter count by a factor of N^2.

L^1 Regularisation (eq. (6.6))

To correct magnitudes of the linear term ψ^θ, the paper adds

$$\tilde{\mathcal{L}}(y_m,\theta_V,\theta_A) \;:=\; \hat{\mathcal{L}}(y_m,\theta_V,\theta_A) \;+\; \beta\,\Big\|\psi^\theta(x)\Big\|, \qquad \beta = 100 \text{ (paper optimal)}. \tag{13}$$

Electronic Trading Application (Section 6)

Asset Price Dynamics (eqs. (6.1)–(6.4))

The mid-price S_t and cumulative transient impact Y_t satisfy

$$\mathrm{d}S_t = \big(\mu(S_t) + g(\nu_t)\big)\,\mathrm{d}t + \mathrm{d}Y_t + \sigma\,\mathrm{d}W_t, \tag{14}$$

$$\mathrm{d}Y_t = -\rho\,Y_t\,\mathrm{d}t + \gamma\,h(\nu_t)\,\mathrm{d}t, \tag{15}$$

where $\bar{\nu}_t = \sum_{i\in\mathcal{N}} \nu_{i,t}$ is the total trading rate. Under the paper's model assumptions:

- Mean-reversion: $\mu(S) = \kappa(\theta - S)$, $\kappa > 0$.
- Linear permanent impact: $g(\nu) = \eta\,\bar{\nu}$.
- Square-root transient impact: $h(\nu) = \mathrm{sign}(\bar{\nu})\sqrt{|\bar{\nu}|}$.

Inventory Dynamics

Each agent i tracks inventory $q_{i,t}$ via

$$q_{i,t} = q_{i,0} + \int_0^t \nu_{i,s}\,\mathrm{d}s. \tag{16}$$

In discrete time with decision points $t_1, \ldots, t_M$ and step ΔT:

$$q_{i,t_{m+1}} = q_{i,t_m} + \nu_{i,t_m}\,\Delta T. \tag{17}$$

Cash Process and Reward (eqs. (6.4)–(6.5))

The cash process is

$$X_{i,t} = -\int_0^T \nu_{i,t}(S_t + b_1\nu_{i,t})\,\mathrm{d}t, \tag{18}$$

and agent i's total expected reward is

$$R_i \;:=\; \mathbb{E}\Big[X_{i,T} \;+\; q_{i,T}(S_T - b_2\,q_{i,T}) \;-\; b_3\int_0^T q_{i,t}^2\,\mathrm{d}t\Big]. \tag{19}$$

The three terms are: (i) total cash at horizon, (ii) terminal liquidation at penalised price, (iii) running urgency penalty proportional to squared holdings.

Paper Parameters (Table 1)

κ	θ	σ	γ	ρ	η	b_1	b_2	b_3
0.1	10	0.01	0.02	0.5	0.05	0.1	0.1	0

$N = 5$ agents, $T = 5$ hours, $M = 10$ decision points, $\Delta T = 0.5$.

State Features (Section 6.1)

The state vector x_t used in experiments comprises four scalars:

$$x_t \;=\; \Big(S_t,\;\; t,\;\; \textstyle\sum_i (q_{i,0} - q_{i,t}),\;\; Y_t\Big)^\top \;\in\; \mathbb{R}^4. \tag{20}$$

Nash-DQN Actor-Critic Algorithm (Algorithm 5.1)

Algorithm 5.1 (Nash-DQN Actor-Critic)

Input: Episodes B, minibatch size $\hat{M}$, game steps N, noise $\{\sigma_b\}_{b=1}^B$
Initialise: Replay buffer $\mathcal{D}$, parameters (θ_A, θ_V)
for episode $b = 1$ **to** B **do**
Reset simulation; observe x_0.
for step $t = 1$ **to** N **do**
Select $u \leftarrow \mu^{\theta_A}(x) + \varepsilon, \quad \varepsilon \sim \mathcal{N}(0, \sigma_b I)$
Observe transition $y_t = (x_{t-1}, u, x_t)$; store in $\mathcal{D}$
Sample $\mathcal{Y} = \{y_i\}_{i=1}^{\hat{M}}$ **from** $\mathcal{D}$
Gradient step on $\frac{1}{\hat{M}+1}\sum_{y\in\mathcal{Y}\cup\{y_t\}} \mathcal{L}(y, \theta_V, \theta_A)$ **over** θ_V
Gradient step on same loss over θ_A
return (θ_A, θ_V)

Key insight. Because $\hat{Q}_i$ is concave in u_i (guaranteed by $P_{11,i} \succ 0$), the Nash operator $\mathrm{Nash}_u\,\hat{Q}$ is bijective and returns $u^* = \mu^\theta(x)$ analytically — no NP-hard inner optimisation is required at each step (in contrast to Hu & Wellman 2003).

Theory-to-Practice Mapping

The table below maps every mathematical object to its implementation role. Students A/B own column 1; Students C/D own columns 3–4.

Math Object	Formula (from paper)	Code / Environment Role	Loss / Network Role
S_t (mid-price)	$\mathrm{d}S_t = \kappa(\theta - S_t)\mathrm{d}t + \eta\bar{\nu}_t\mathrm{d}t + \mathrm{d}Y_t + \sigma\mathrm{d}W_t$	`env.S` — updated each step using Euler-Maruyama; $\Delta S \sim \mathcal{N}(0, \sigma^2\Delta T)$ plus drift	Input to reward; *not* a network input
Y_t (transient impact)	$Y_{t+\Delta T} = (1-\rho\Delta T)Y_t + \gamma\,\mathrm{sign}(\bar{\nu})\sqrt{\lvert\bar{\nu}\rvert}\,\Delta T$	`env.Y` — 4th component of state vector x_t	Network input; drives μ^θ and P^θ
$q_{i,t}$ (inventory)	$q_{i,t+\Delta T} = q_{i,t} + \nu_{i,t}\Delta T$	`env.inv[i]` — clipped to valid range; terminal penalty if $q_{i,T} \neq 0$	State input; also enters reward r_i
$\bar{\nu}_t = \sum_i \nu_{i,t}$ (total flow)	Observable to all agents; 3rd state feature	`env.total_flow` — accumulated since $t = 0$	State input
$x_t \in \mathbb{R}^4$ (state)	$(S_t,\, t,\, \sum_i(q_{i,0} - q_{i,t}),\, Y_t)^\top$ (eq. (20))	`numpy` array shape $(4,)$; input to both networks	Input dim of $\hat{V}^\theta$ and $\mathcal{A}^\theta$
$\nu_{i,t}$ (action)	$\nu_{i,t} \in \mathbb{R}$ (buy/sell rate)	`agent.act(x)` — outputs $\mu_i^\theta(x) + \varepsilon$ during training; $\mu_i^\theta(x)$ at eval	μ_i^θ is a direct network output
r_i (step reward)	$-\nu_{i,t}(S_t + b_1\nu_{i,t})\Delta T - b_3 q_{i,t}^2\Delta T$ plus terminal term	`env.step()` return value; stored in replay buffer	Appears as $r(x_m; u_m)$ in loss (10)
$\hat{V}^\theta(x)$ (value net)	Separate network, 4 hidden layers × 32 nodes, SiLU	`ValueNet(x)` $\to \mathbb{R}^N$	Appears as $\hat{V}^\theta(x_m)$ and $\hat{V}^\theta(x'_m)$ in loss
$\mu_i^\theta(x)$ (Nash policy)	Output of main network; direct Nash action	`ActorNet(x)` $\to \mu_i$ per agent	$u^* = \mu^\theta$ used to compute target $\gamma_i\hat{V}^\theta(x')$
$P^\theta_{11,i}(x)$	Lower-triangular Cholesky $L_{11}L_{11}^\top \succ 0$	`CholeskyLayer`: outputs L; reconstructs $P_{11} = LL^\top$	Enforces $\mathcal{A}_i$ concave in u_i throughout training
$\mathcal{L}(\theta)$ (total loss)	$\frac{1}{M}\sum_m \lVert\hat{V}(x_m) + \mathcal{A}(x_m; u_m) - r_m - \gamma\hat{V}(x'_m)\rVert^2$	`F.mse_loss(Q_pred, target.detach())`	Minimised jointly over θ_V, θ_A
$\beta\lVert\psi^\theta(x)\rVert$ (L1 reg.)	Added to loss; $\beta = 100$	`loss += beta * psi.abs().sum()`	Prevents linear term from dominating
Replay buffer $\mathcal{D}$	Stores $y_m = (x_m, u_m, x'_m)$	`deque(maxlen=N)`; `random.sample(batch)`	Decorrelates gradient updates
Permutation-invariant layer	$f_{\mathrm{inv}}(z) = \sigma\left(\sum_j \phi(z_j)\right)$ (eq. (4.10))	`PermInvLayer`: sums ϕ-outputs over agent dim	Encodes label-invariance; reduces params by N^2

Technical Specifications for 2-Agent Simplified Implementation

The project uses $N = 2$ agents as a tractable starting point. The full paper uses $N = 5$; the 2-agent case preserves all structural features.

State Vector

$$x_t \;=\; \Big(S_t,\;\; t/T,\;\; (q_{1,0} - q_{1,t}) + (q_{2,0} - q_{2,t}),\;\; Y_t\Big)^\top \;\in\; \mathbb{R}^4, \tag{21}$$

with normalisation: S_t/S_0, inventories divided by $q_{i,0}$, $Y_t/Y_{\max}$.

Action Space

$\nu_i \in \mathbb{R}$ (continuous). During training: $\nu_i = \mu_i^\theta(x) + \varepsilon$, $\varepsilon \sim \mathcal{N}(0, \sigma_b^2)$ with $\sigma_b \searrow 0$ over episodes. At evaluation: $\nu_i = \mu_i^\theta(x)$ (greedy).

Network Architecture

Advantage network $\mathcal{A}^{\theta_A}$: permutation-invariant layer (3 hidden layers × 20 nodes, SiLU) → main network (4 hidden layers × 32 nodes, SiLU) → outputs $(\mu_i, L_{11,i}, P_{12,i}, P_{22,i}, \psi_i)$.

Value network $\hat{V}^{\theta_V}$: standard MLP, 4 hidden layers × 32 nodes, SiLU → scalar per agent.

Q-Value Computation

$$\hat{Q}_i(x; u_i, u_{-i}) = \hat{V}_i(x) + \mathcal{A}_i^\theta(x; u_i, u_{-i}), \tag{22}$$

$$\begin{aligned}\text{with}\quad \mathcal{A}_i = &-(u_i - \mu_i)^\top P_{11,i}(u_i - \mu_i) - (u_i - \mu_i)^\top P_{12,i}(u_j - \mu_j) \\ &- (u_j - \mu_j)^\top P_{22,i}(u_j - \mu_j) + (u_j - \mu_j)^\top \psi_i.\end{aligned}$$

Training Loss

$$\mathcal{L}(\theta_V, \theta_A) \;=\; \frac{1}{M}\sum_{m=1}^{M} \Big\| \hat{V}^{\theta_V}(x_m) + \mathcal{A}^{\theta_A}(x_m; u_m) - r(x_m; u_m) - \gamma\, \hat{V}^{\theta_V}(x'_m) \Big\|^2 + \beta \left\|\psi^{\theta_A}(x_m)\right\|. \tag{23}$$

Update alternately: gradient step over θ_V, then over θ_A.

Stability Specifications

Hyperparameter	Value / Rule
Optimiser	Adam, weight decay 0.001
Learning rate	0.003 (paper value)
Mini-batch size	10 full episodes
Max training iterations	20 000
Early stopping	No improvement for 3 000 iterations
Exploration noise σ_b	Decaying schedule; e.g. $\sigma_b = \sigma_0 e^{-b/\tau}$
L1 regularisation β	100 (tuned on 1 000 random simulations)
Replay buffer size	100 000 transitions
P_{11} enforcement	Cholesky: $P_{11} = LL^\top$, L lower-triangular
Gradient clipping	Norm ≤ 1.0

Four-Week Detailed Roadmap

Week 1 — Model Formulation & Environment Foundation

Goal: Theory group completes mathematical write-up of Sections 2–3 of the paper; Practice group ships a working, unit-tested simulator.

Student A — Theory

- Read Sections 2–3 of arXiv:1904.10554v2 in full with notation log.
- Derive the stochastic game setup: state space $\mathcal{X}$, action spaces $\mathcal{U}_i$, transition kernel p.
- Formally state and prove the form of R_i in eq. (1); verify Markov property.
- Derive the SDE system (eqs. (14)–(15)) and Euler-Maruyama discretisation with error $O(\Delta T)$.
- Write LaTeX document: Sections 2–3 of report (model setup, Nash definition, Bellman eq.).

Student B — Theory

- Study the inventory dynamics (eq. (17)) and cash process; relate to Almgren–Chriss (2001) as single-agent benchmark.
- Derive the reward functional (eq. (19)) from first principles; identify the three terms economically.
- Verify that reward is concave in ν_i given fixed ν_{-i} (required by paper assumptions).
- Survey Casgrain & Jaimungal (2018, 2020) mean-field baseline; note similarities and differences.
- Write LaTeX: Section on model assumptions and parameter Table 1 reproduction.

Student C — Practice

- Set up Python repo: `numpy`, `torch`, `matplotlib`, `gymnasium`.
- Implement `TradingEnv` class: `reset()`, `step()`, `render()`.
- Code Euler-Maruyama updates for S_t, Y_t, $q_{i,t}$ exactly matching eqs. (14)–(17).
- Implement reward per eq. (19) with terminal penalty for $q_{i,T} \neq 0$.
- Unit test: single agent, $\kappa = \rho = \eta = 0$ — verify S_t is pure Brownian motion.

Student D — Practice

- Set up `model.py`: skeleton networks (random-output) for $\hat{V}^{\theta_V}$ and $\mathcal{A}^{\theta_A}$.
- Implement replay buffer: `namedtuple('Transition', ['x','u','r','x_next','done'])`.
- Build dummy data collection loop: 1 000 steps, verify tensor shapes $(B, 4)$, $(B, 1)$, $(B, 1)$.
- Implement permutation-invariant layer following eq. (4.10): $f_{\text{inv}}(z) = \sigma(\sum_j \phi(z_j))$.
- Verify label-invariance numerically: permute agent inputs, confirm identical f_{inv} outputs.

End of Week 1. Theory group delivers LaTeX document covering model setup, SDE system, Nash definition (eqs. (1)–(5)). Practice group delivers `TradingEnv` passing all unit tests. **Joint review:** Student A reads `env.step()` and verifies that the reward formula in code matches eq. (19) exactly. Student C reads Student A's SDE notes to understand the Y_t discretisation in the state update.

✓ WEEK DELIVERABLES

- **A+B**: LaTeX Sections 1–2 (model, Nash definition, Bellman equation, reward).
- **C**: `env.py` — `TradingEnv` with SDE dynamics, passing unit tests.
- **D**: `buffer.py`, `permutation_inv.py` with numerical invariance test.
- **Joint**: State-reward formula cross-validated between LaTeX and code.

Week 2 — Q-Function Architecture & Baseline

Goal: Derive the LQ advantage function structure; implement full network architecture; establish FP/TWAP baselines.

Student A — Theory

- Derive the advantage function decomposition (eqs. (6)–(7)) with full matrix notation.
- Prove that $\mathcal{A}_i^\theta$ is concave in u_i iff $P_{11,i}^\theta \succ 0$; show Cholesky enforcement.
- Derive the closed-form Nash equilibrium $\mu^\theta(x)$ (eq. (9)) from the quadratic structure.
- Prove the label-invariant simplification (eq. (12)) for $N = 2$, $d = 1$. Compute exact parameter count reduction.

Student B — Theory

- Derive the loss function (10)–(11) from the Nash-Bellman equation (5).
- Interpret the loss: it measures violation of Nash-Bellman; minimising it drives $\hat{Q}^\theta \to Q^*$.
- Solve the single-agent Almgren–Chriss problem as closed-form baseline (no RL needed); implement analytically.
- Study Fictitious Play (Algorithm 6.1 in paper) as second baseline; write pseudocode.

Student C — Practice

- Extend `TradingEnv` to 2-agent joint actions: `step([nu1, nu2])`.
- Implement TWAP baseline agent: $\nu_i = q_{i,0}/T$ constant.
- Run 500 TWAP episodes; compute mean/std of total reward per agent.
- Implement Fictitious Play (Algorithm 6.1): alternating best-response, using TWAP as initial policy.

Student D — Practice

- Implement `AdvantageNet`: permutation-invariant layer → main network → outputs $(\mu_i, L_{11,i}, P_{12,i}, P_{22,i}, \psi_i)$.
- Implement `ValueNet`: MLP → scalar $\hat{V}_i$ per agent.
- Implement `Q_value(x, u1, u2)` using eq. (7) with Cholesky $P_{11} = LL^\top$.
- Unit test: verify $\hat{Q}_i$ is concave in u_i by plotting $\hat{Q}_i$ vs. u_i for random θ.

End of Week 2 — Critical Checkpoint. Student D runs the Almgren–Chriss closed-form solution (from B) through `TradingEnv` and confirms rewards match analytical expectations. Student A reads `Q_value()` in code and verifies it matches eq. (7) exactly, including matrix dimensions. This is the most important cross-validation of the project.

✓ WEEK DELIVERABLES

- **A+B**: LaTeX Section 3 (LQ decomposition, advantage form, loss derivation).
- **B**: Almgren–Chriss closed-form baseline in `benchmark.py`.
- **C**: 2-agent `TradingEnv`; TWAP and FP baselines benchmarked.
- **D**: Full `AdvantageNet` and `ValueNet` with concavity unit test passing.

Week 3 — Nash-DQN Training Loop & Convergence

Goal: Full Algorithm 5.1 running end-to-end; training stable; first numerical results.

Student A — Theory

- Literature review: convergence conditions for Nash Q-learning (Hu & Wellman 2003, Theorem 4.1).
- Define the Nash gap $G(x) := \max_{u_i} \hat{Q}_i(x; u_i, u^*_{-i}(x)) - \hat{Q}_i(x; u^*_i(x), u^*_{-i}(x)) \geq 0$.
- Interpret training curves economically: declining TD-error $\Leftrightarrow$ Bellman consistency improving.
- Write Section 4 of report: convergence discussion, Nash gap analysis, discussion of LQ approximation quality.

Student B — Theory

- Analyse the L^1 regularisation (eq. (13)): why does large ψ^θ harm training? Provide mathematical intuition.
- Derive the price of anarchy: bound on social welfare loss $(R_1^* + R_2^*) - (R_1^{\text{Nash}} + R_2^{\text{Nash}})$.
- Prove: if $\kappa = \eta = 0$ (no interaction), Nash-DQN decouples into two independent single-agent problems.
- Write comparison: learned Nash-DQN policy vs. Almgren–Chriss vs. FP (economic narrative).

Student C — Practice

- Implement `train.py`: full Algorithm 5.1 with alternating θ_V, θ_A gradient steps.
- Add Gaussian exploration noise: $\nu_i = \mu_i^\theta(x) + \varepsilon$, decaying σ_b.
- Log per-iteration: loss $\mathcal{L}$, Nash gap $G(x)$, episode reward per agent, $q_{i,T}$.
- Run 5 000 iterations; verify loss decreases and $G(x) \to 0$.

Student D — Practice

- Implement L^1 regulariser on ψ^θ; sweep $\beta \in \{10, 100, 1000\}$; confirm $\beta = 100$ minimises loss (reproducing Figure 1 of paper).
- Implement evaluation loop: 1 000 episodes, greedy policy, compute mean and std of R_i.
- Plot $\nu_i(t)$ vs. TWAP vs. Almgren–Chriss: inventory trajectory across time steps.
- Begin heatmap: $\nu_i(t, q_i, S, Y)$ at fixed $Y \in \{-0.2, 0, 0.2\}$ (reproducing Figure 2 of paper).

End of Week 3 — Numerical Results Review. All four students review training curves together. Key questions: (1) Does learned $\nu_i(t)$ outperform TWAP in mean reward? (2) Is Nash gap $G(x) \approx 0$ at convergence? (3) Does Student B's analytical comparison match numerical plots? If training is diverging, execute the Emergency Protocol in Section 7.

✓ WEEK DELIVERABLES

- **C**: `train.py` — Algorithm 5.1, stable over 5 000+ iterations.
- **D**: β sweep reproducing Figure 1; evaluation loop; begin Figure 2 heatmap.
- **A+B**: LaTeX Sections 4–5 (convergence, Nash gap, economic interpretation).
- **All**: Numerical comparison — Nash-DQN vs. TWAP vs. Almgren–Chriss.

Week 4 — Evaluation, Figures & Report

Goal: Reproduce Figures 1–3 of the paper; complete written report; prepare presentation.

Student A — Theory

- Write final report Sections 1–2: introduction, model, all LaTeX equations (1)–(13).
- Write Section 6: conclusions, limitations (LQ approximation quality, $N = 2$ vs. $N = 5$), future work.
- Proofread complete document for mathematical consistency; check all equation labels.
- Prepare 4–5 mathematical slides: Nash equilibrium, advantage function, loss, Nash operator.

Student B — Theory

- Write report Sections 3–5: theory (Nash-Bellman, LQ form), algorithm, results analysis.
- Verify all figures have rigorous captions explaining economic significance.
- Deviations experiment (Figure 3 of paper): evaluate scenario (i) Nash-DQN vs. FP agents and (ii) FP vs. Nash-DQN agents.
- Run 100 repetitions $\times$ 1 000 simulations; plot reward distributions as histograms.

Student C — Practice

- Full hyperparameter sweep: lr $\in \{3 \times 10^{-3}, 10^{-3}, 3 \times 10^{-4}\}$, $b_3 \in \{0, 0.01, 0.1\}$.
- Reproduce Figure 2: heatmaps $\nu_i^*(t, q_i)$ at $Y \in \{-0.2, 0, 0.2\}$ for Nash-DQN.
- Reproduce Figure 3: reward deviation histograms (100 repeats $\times$ 1 000 paths per scenario).
- Ablation: $\eta = 0$ (no permanent impact) — verify policy collapses to Almgren–Chriss.

Student D — Practice

- Polish all figures: consistent colourmap, axis labels, captions matching paper style.
- Run 500-episode stress test: compute 95% confidence intervals on R_i.
- Build 10-slide presentation; prepare live training demo (show loss curve converging in real time).
- Clean GitHub repo: `README.md` with install instructions, parameter table, results summary.

End of Week 4 — Final Submission. Complete report submitted; all figures reproduced with statistical confidence intervals. Presentation rehearsed to 15 minutes. Code repo public and documented.

✓ WEEK DELIVERABLES

- **A+B**: Full report (15–20 pages); all equations in LaTeX; economic narrative.
- **C**: Figures 1–3 reproduced; ablation studies complete.
- **D**: Publication-quality plots; 500-episode confidence intervals; clean repo.
- **All**: 10-slide presentation; 15-minute rehearsed demo.

Theory vs Practice Group Responsibilities

Theory Group (A + B) OWNS

- All mathematical derivations, Sections 1–4 above.
- Proof of Nash eq. existence (LQ game).
- Closed-form Almgren–Chriss benchmark.
- L^1 regularisation motivation.
- Complete written report and LaTeX.
- Validation: code formulas match paper equations.
- Economic interpretation of every result.

Practice Group (C + D) OWNS

- `env.py`: SDE dynamics, reward, done.
- `buffer.py`: replay buffer.
- `model.py`: $\hat{V}$, $\mathcal{A}$, Cholesky layer, perm-inv.
- `train.py`: Algorithm 5.1 end-to-end.
- All plots and figure reproduction.
- Hyperparameter sweep and ablations.
- GitHub repo and README.

Recommended Resources

For Theory Students (A & B)

- **The paper**: Casgrain, Ning & Jaimungal (2022), arXiv:1904.10554v2 — read all sections including appendix.
- **Almgren & Chriss** (2001): "Optimal Execution of Portfolio Transactions", *Journal of Risk* — foundational single-agent baseline; master before Week 2.
- **Cartea, Jaimungal & Peñalva** (2015): *Algorithmic and High-Frequency Trading*, Cambridge — Chapters 6–8 for stochastic control and optimal execution.
- **Øksendal** (2003): *Stochastic Differential Equations* (6th ed.) — Chapters 4–5 (Itô calculus), Chapters 7–11 (stochastic control).
- **Hu & Wellman** (2003): "Nash Q-Learning for General-Sum Stochastic Games", *JMLR* — theoretical basis of Nash-DQN; read Theorem 4.1 on convergence.
- **Casgrain & Jaimungal** (2020): "Mean-Field Games with Differing Beliefs", *Mathematical Finance* — context for mean-field limit.

For Practice Students (C & D)

- **Mnih et al.** (2015): "Human-level control through DRL", *Nature* — original DQN; replay buffer and target network concepts.
- **Sutton & Barto** (2018): *RL: An Introduction* (2nd ed., free online) — Chapter 6 (TD), Chapter 9 (function approximation).
- **Gu et al.** (2016): "Continuous Deep Q-Learning with Model-Based Acceleration" (ICML) — iLQG framework that Nash-DQN extends.
- **Lowe et al.** (2017): "Multi-Agent Actor-Critic for Mixed Cooperative-Competitive Environments" (NeurIPS) — MADDPG, closely related architecture.

- **Zaheer et al.** (2017): "Deep Sets" (NeurIPS) — mathematical foundation for permutation-invariant layers (eq. (4.10) of paper).
- **PyTorch docs**: `nn.Module`, `optim.Adam`, `autograd` — write custom training loop from scratch; no Stable-Baselines.

Grading Checklist & Risk Mitigation

Mathematical Rigour Checklist

1. All mathematical objects defined with notation consistent with arXiv:1904.10554v2.
2. Nash equilibrium proven to exist in the 2-agent LQ game (not just cited).
3. Advantage function derivation with Cholesky enforcement of $P_{11} \succ 0$.
4. Loss function derived from Nash-Bellman equation (5) step-by-step.
5. Closed-form Almgren–Chriss benchmark solved analytically and verified numerically.
6. Nash gap $G(x) \geq 0$ defined, computed, and shown to decrease during training.
7. Label-invariance reduction (eq. (12)) proven for $N = 2$.
8. L^1 regularisation (eq. (13)) motivated mathematically; β tuned as in paper.
9. Economic interpretation given for every figure — not just "loss decreased".
10. Deviations experiment (Figure 3) interpreted: inability to reject null means Nash-DQN $\approx$ FP in quality but is more data-efficient.

Computational Strength Checklist

1. `env.step()` reward formula validated against Student A's LaTeX before training.
2. $P^{\theta}_{11,i} \succ 0$ enforced at all times via Cholesky; verified by eigenvalue check.
3. Alternating optimisation over θ_V and θ_A matches Algorithm 5.1 exactly.
4. $\beta = 100$ for L^1 regularisation reproduced by own sweep (Figure 1 reproduced).
5. Exploration noise decays monotonically; greedy at evaluation.
6. Nash gap approaches zero; plotted alongside loss per agent.
7. Mean reward of Nash-DQN $\geq$ TWAP (key empirical result to verify).
8. Deviation experiment: 100 repeats $\times$ 1 000 paths per scenario (Figure 3 reproduced).
9. All code committed to a clean repo; `README.md` with full parameter table.

Common Pitfalls

! COMMON PITFALLS

P1. Not alternating updates: Algorithm 5.1 requires alternating gradient steps over θ_V then θ_A — joint update destabilises training.

P2. P_{11} not positive-definite: if raw network outputs are used without Cholesky, $\hat{Q}_i$ is not concave in u_i and the Nash operator is undefined.

P3. Wrong state features: paper uses $(S_t, t, \sum_i (q_{i,0} - q_{i,t}), Y_t)$ (eq. (20)), not individual inventories — agents observe total order flow, not each other's inventory.

P4. Missing L^1 regularisation: training without $\beta\|\psi^{\theta}\|$ produces suboptimal policies (Figure 1 of paper explicitly demonstrates this).

P5. Reward scale: with $S_0 \approx 10$, rewards are $O(1)$–$O(10)$; if unnormalised, Q-values diverge.

P6. Terminal condition: agents trade over $[0, T]$; $q_{i,T}$ need not be zero — the terminal penalty $b_2 q^2_{i,T}$ is the enforcement mechanism.

P7. Confusing γ (impact) vs. γ_i (discount): the paper uses γ for the transient impact coefficient (eq. (15)) and γ_i for the discount rate (eq. (1)) — keep these distinct in code.

P8. Square-root impact: $h(\nu) = \text{sign}(\bar{\nu})\sqrt{|\bar{\nu}|}$ is non-smooth at $\bar{\nu} = 0$; handle numerically with a small epsilon: $h(\nu) \approx \text{sign}(\bar{\nu})\sqrt{|\bar{\nu}| + \epsilon}$.

Emergency Divergence Protocol

! EMERGENCY DIVERGENCE PROTOCOL

Step 1. Reduce learning rate by $10\times$ (try 3×10^{-4}). Fixes 80% of divergence cases.
Step 2. Check $P_{11} \succ 0$: print minimum eigenvalue of P_{11} each epoch; if ≤ 0, Cholesky is broken.
Step 3. Add gradient clipping: `torch.nn.utils.clip_grad_norm_(params, 1.0)`.
Step 4. Increase β: try $\beta = 1000$; large ψ is a common early divergence cause.
Step 5. Test single-agent first: set $N = 1$, $\eta = \rho = \gamma = 0$; verify agent learns Almgren–Chriss. If this fails, the environment is wrong.
Step 6. Reduce to $\mathcal{A}$ only: remove $\hat{V}^\theta$, set $\hat{V} = 0$; verify advantage function alone can learn.
Step 7. Fallback: if Nash-DQN fails to converge, present the Fictitious Play baseline (Algorithm 6.1) with DDPG inner loop as the main result and discuss Nash-DQN analytically. This still demonstrates multi-agent equilibrium learning and satisfies the project requirements.